

- 人工智能实验报告 第8周
 - 一.实验题目：中药图片分类任务
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

人工智能实验报告 第8周

姓名:陈安康 学号:22320021

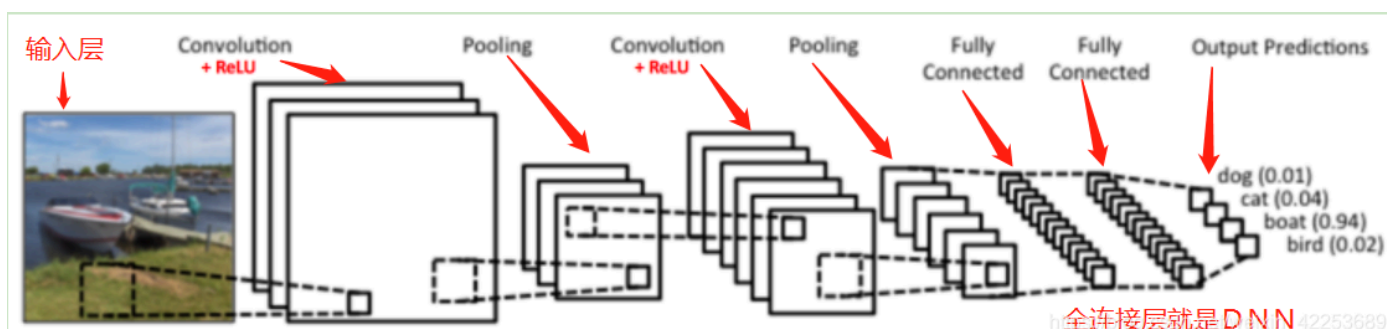
一.实验题目：中药图片分类任务

二.实验内容

1.算法原理

卷积神经网络通过对图像通过卷积操作进行特征提取、激活、特征压缩等一系列操作来实现学习，优化网络中的参数权重，进而完成分类等复杂任务。

在结构上，卷积神经网络包括输入层、卷积层、激活层、池化层、和全连接层。其中卷积层、激活层、池化层分别用来进行卷积操作提取特征、输入激活函数进行激活来引入非线性、压缩特征以缩小数据量。全连接层则是将多维的数据拉伸成一维向量从而进行全连接，输出结果。卷积层、激活层、池化层不断循环堆叠，对数据进行不断的提取、激活、压缩操作，以逐渐提取更高级别的特征，进而得到结果。



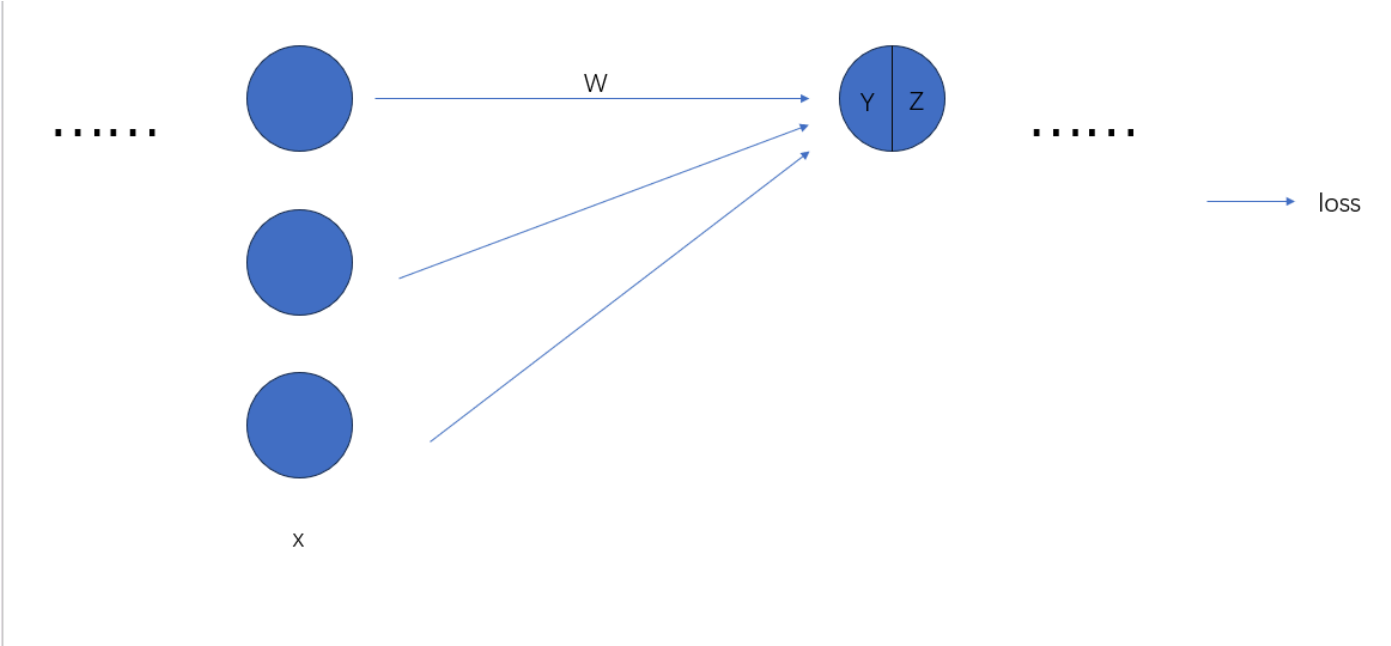
对于卷积层，通过规定好的卷积核依据规定好的步长在图像上一边移动，一边计算区域得分，从而得到一个代表图像中每一部分的得分的矩阵。此矩阵通过激活函数进行激活，引入非线性部分，进而使网络能够学习复杂的特征。接着这个矩阵通过池化层来减小特征图的大小来减少计算复杂性。它通过选择池化窗口内的最大值或平均值来实现。这有助于提取最重要的特征。

最后经过不断地堆叠卷积层、激活层、池化层，提取更高级别的特征，然后将这些特征拉伸成一维向量，通过全连接层将提取的特征映射转化为网络的最终输出。

参数优化上，采用的是神经网络常用的反向传播算法（BP算法）。此算法建立在梯度下降算法的基础之上，有前向传播与反向传播两阶段。

依据梯度下降算法的理论，我们优化的目标是最小化损失函数值。

假设某一中隐藏层的输入为 x ，权重为 w ，输出为 z ，最终误差为 $loss$ ，激活函数为 $f(x)$ 。则会有以下关系：



$$z = f(y), \quad y = \sum_{i=1}^n w_i * x_i + b$$
$$\frac{\partial loss}{\partial w_j} = \frac{\partial loss}{\partial z} * \frac{\partial z}{\partial y} * \frac{\partial y}{\partial w_j}$$

由此我们只要知道各个偏函数即可。在前向传播过程中，后两个偏导数可以随着数据在网络中的前向传递算出来，对于 $loss$ 对于 z 的偏导数，由于这只是其中一个隐藏层，无法直接算出 $loss$ 对 z 的偏导数，所以通过算出最终的损失值，接着反向推导运用链式法则逐步求出 $loss$ 对 z 的偏导，从而得到 $loss$ 对某一权重的梯度，接着便可以像梯度下降法一样优化参数。

2.关键代码展示(可选)

构建的CNN模型如下，一共有三层堆叠。

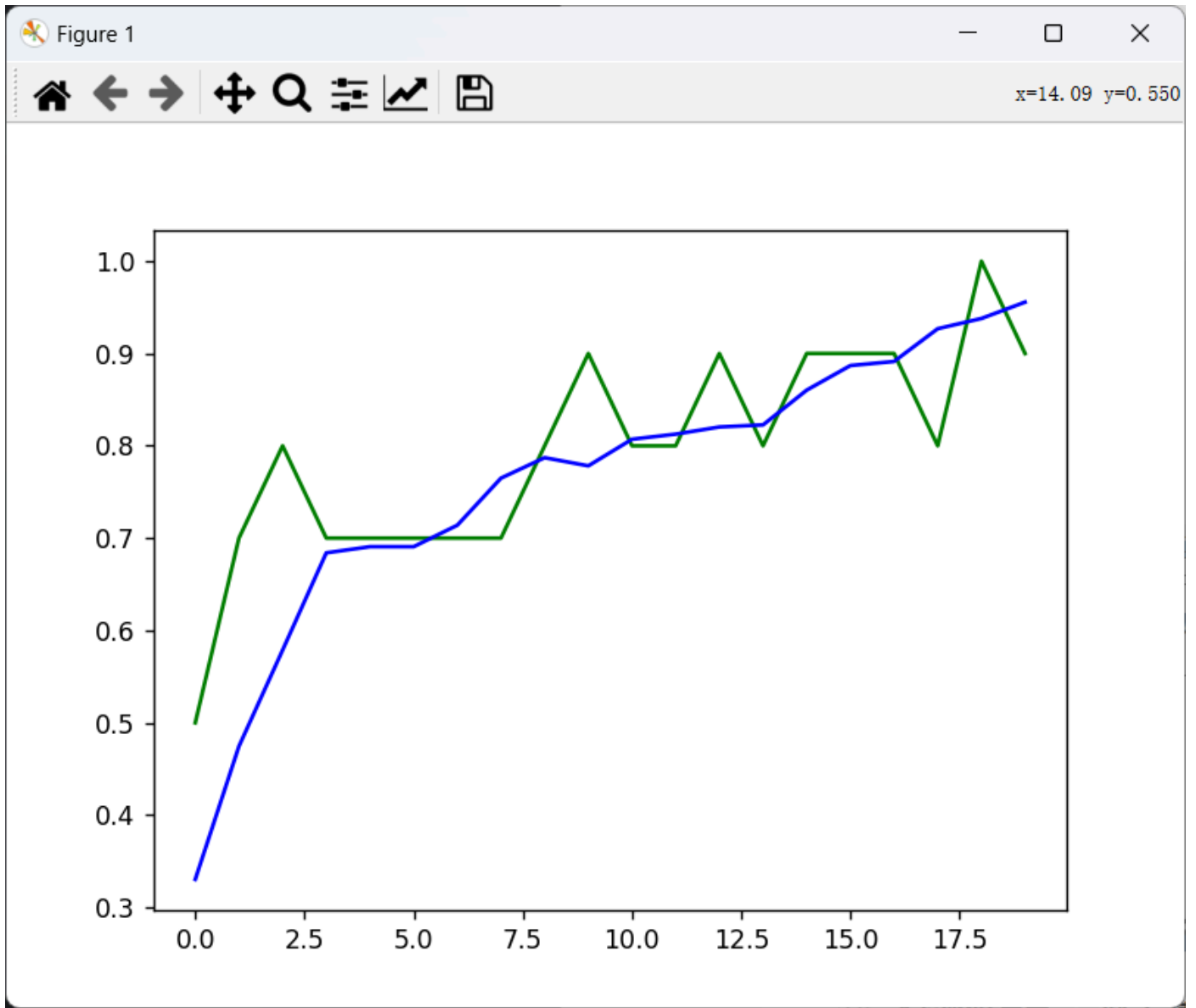
```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels= 3,
                out_channels= 20,
                kernel_size= 5,
                stride= 1,
                padding= 1,
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size= 2),
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(20, 40, 4, 1, 1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.conv3 = nn.Sequential(
            nn.Conv2d(40, 80, 4, 1, 1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.out = nn.Linear(80 * 27 * 27, 5)
```

三.实验结果及分析

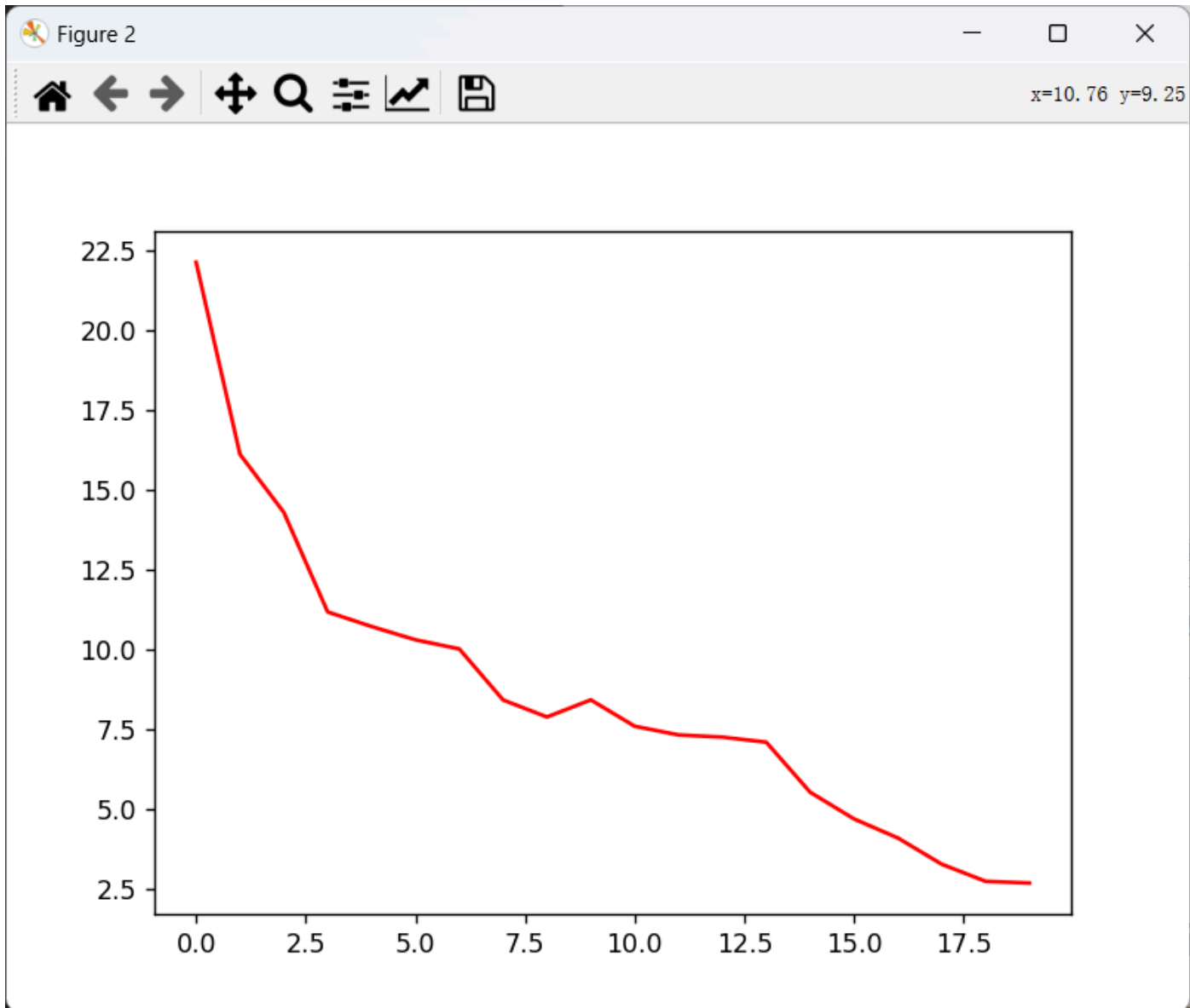
1.实验结果展示示例（可截图可表可文字，尽量可视化）

以下是结果展示，可以看到随着迭代次数增加，预测的准确率上升趋势。**loss**曲线呈下降趋势。

预测准确率图，蓝色线为训练集的准确率，绿色线为测试集的准确率



loss曲线图:

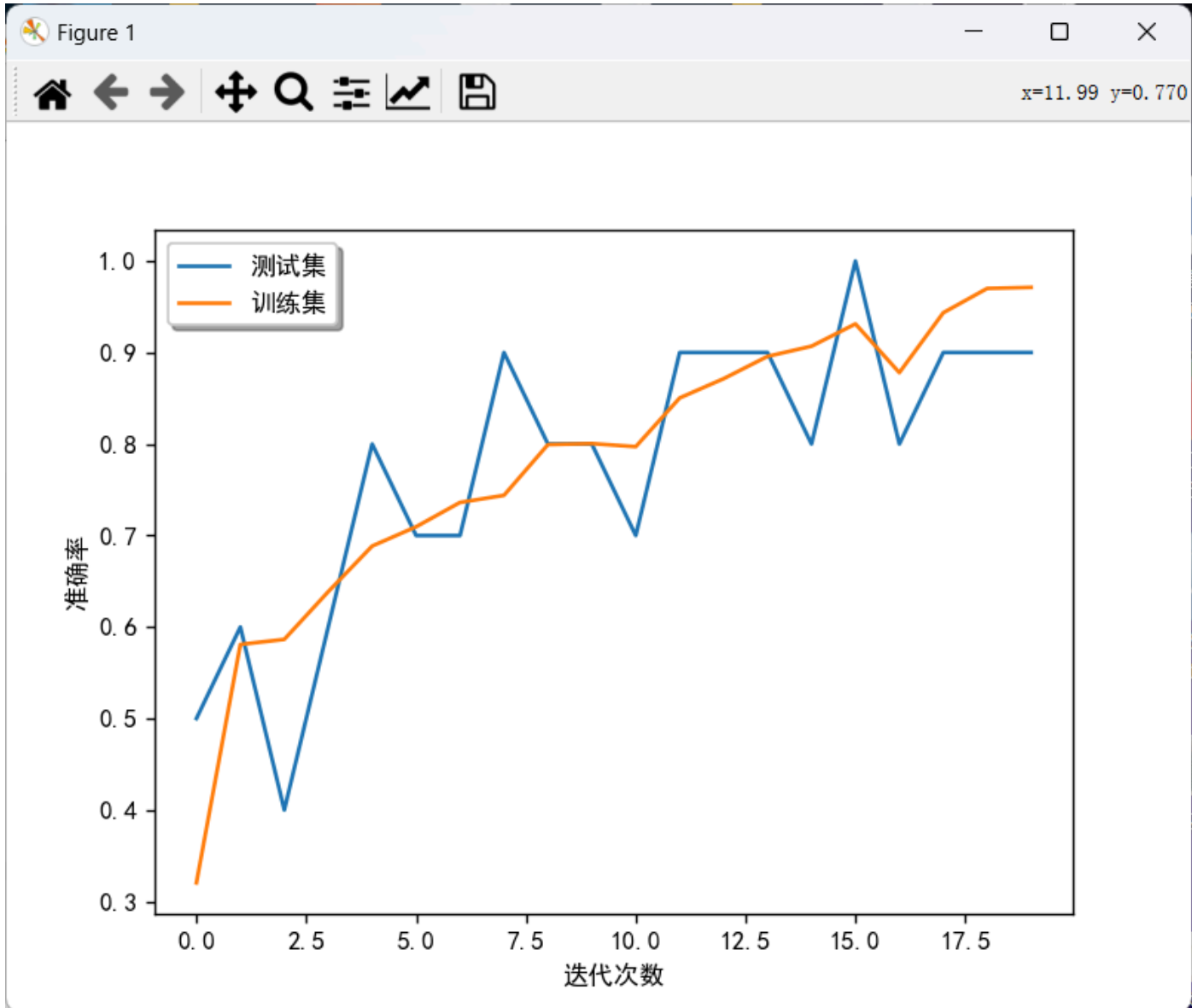


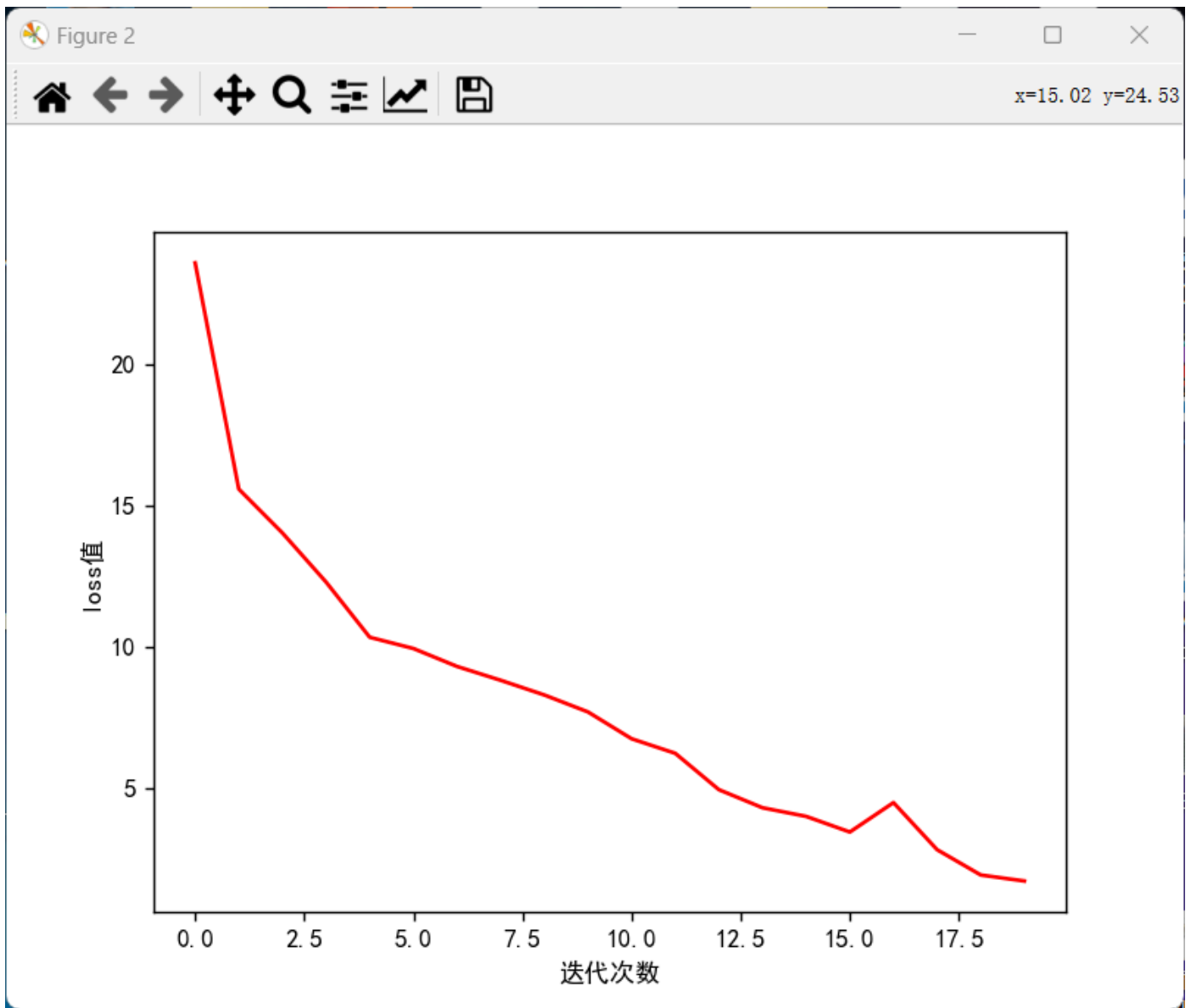
测试准确率	:	Epoch: 0		train loss: 22.1180		test accuracy: 0.50
训练准确率	:	Epoch: 0		train loss: 22.1180		train accuracy: 0.33
测试准确率	:	Epoch: 1		train loss: 16.1097		test accuracy: 0.70
训练准确率	:	Epoch: 1		train loss: 16.1097		train accuracy: 0.47
测试准确率	:	Epoch: 2		train loss: 14.2908		test accuracy: 0.80
训练准确率	:	Epoch: 2		train loss: 14.2908		train accuracy: 0.58
测试准确率	:	Epoch: 3		train loss: 11.1738		test accuracy: 0.70
训练准确率	:	Epoch: 3		train loss: 11.1738		train accuracy: 0.68
测试准确率	:	Epoch: 4		train loss: 10.7182		test accuracy: 0.70
训练准确率	:	Epoch: 4		train loss: 10.7182		train accuracy: 0.69
测试准确率	:	Epoch: 5		train loss: 10.3025		test accuracy: 0.70
训练准确率	:	Epoch: 5		train loss: 10.3025		train accuracy: 0.69
测试准确率	:	Epoch: 6		train loss: 10.0171		test accuracy: 0.70

训练准确率	:	Epoch: 8		train loss: 7.8857		train accuracy: 0.79
测试准确率	:	Epoch: 9		train loss: 8.4219		test accuracy: 0.90
训练准确率	:	Epoch: 9		train loss: 8.4219		train accuracy: 0.78
测试准确率	:	Epoch: 10		train loss: 7.5962		test accuracy: 0.80
训练准确率	:	Epoch: 10		train loss: 7.5962		train accuracy: 0.81
测试准确率	:	Epoch: 11		train loss: 7.3247		test accuracy: 0.80
训练准确率	:	Epoch: 11		train loss: 7.3247		train accuracy: 0.81
测试准确率	:	Epoch: 12		train loss: 7.2560		test accuracy: 0.90
训练准确率	:	Epoch: 12		train loss: 7.2560		train accuracy: 0.82
测试准确率	:	Epoch: 13		train loss: 7.0979		test accuracy: 0.80
训练准确率	:	Epoch: 13		train loss: 7.0979		train accuracy: 0.82
测试准确率	:	Epoch: 14		train loss: 5.5324		test accuracy: 0.90
训练准确率	:	Epoch: 14		train loss: 5.5324		train accuracy: 0.86

测试准确率	:	Epoch: 14		train loss: 5.5324		test accuracy: 0.90
训练准确率	:	Epoch: 14		train loss: 5.5324		train accuracy: 0.86
测试准确率	:	Epoch: 15		train loss: 4.6964		test accuracy: 0.90
训练准确率	:	Epoch: 15		train loss: 4.6964		train accuracy: 0.89
测试准确率	:	Epoch: 16		train loss: 4.1002		test accuracy: 0.90
训练准确率	:	Epoch: 16		train loss: 4.1002		train accuracy: 0.89
测试准确率	:	Epoch: 17		train loss: 3.2792		test accuracy: 0.80
训练准确率	:	Epoch: 17		train loss: 3.2792		train accuracy: 0.93
测试准确率	:	Epoch: 18		train loss: 2.7431		test accuracy: 1.00
训练准确率	:	Epoch: 18		train loss: 2.7431		train accuracy: 0.94
测试准确率	:	Epoch: 19		train loss: 2.6888		test accuracy: 0.90
训练准确率	:	Epoch: 19		train loss: 2.6888		train accuracy: 0.96

以下是另一次实验的结果:





```
测试准确率 : Epoch: 17 | train loss: 2.8536 | test accuracy: 0.90
训练准确率 : Epoch: 17 | train loss: 2.8536 | train accuracy: 0.94
测试准确率 : Epoch: 18 | train loss: 1.9593 | test accuracy: 0.90
训练准确率 : Epoch: 18 | train loss: 1.9593 | train accuracy: 0.97
测试准确率 : Epoch: 19 | train loss: 1.7466 | test accuracy: 0.90
训练准确率 : Epoch: 19 | train loss: 1.7466 | train accuracy: 0.97
```

2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

评测指标包括准确率，混淆矩阵。

准确率来说，训练后期，如实验结果展示，测试集准确率在80% - 100%，训练集准确率在95%左右。准确率上十分理想。

上面的实验结果的训练集结果的混淆矩阵如下（因为测试集数据太少了，分析的意义不大，所以展示训练集的混淆矩阵）：


```
[[178.  1.  0.  0.  1.]
 [  4. 181.  0.  3.  2.]
 [  0.  0. 185.  0.  0.]
 [  3.  5.  0. 156.  3.]
 [  1.  1.  0.  2. 176.]]
```

为了方便分析，转换成图表如下（类别用0-4来表示）：

		预 测				
		0	1	2	3	4
实 际	0	178	1	0	0	1
	1	4	181	0	3	2
	2	0	0	185	0	0
	3	3	5	0	156	3
	4	1	1	0	2	176

可以看到，大部分的数据集中在对角线上。

经过计算，可以得到每一个分类预测的精确率（表示模型正确预测为正类别的样本数量与表示模型错误地将负类别样本预测为正类别的样本数量和模型正确预测为正类别的样本数量的比值，公式为 $Precision = \frac{True\ Positive}{True\ Positive + False\ Positive}$ ）分别为：

0: 95.69%

1: 96.27%

2: 100%

3: 96.89%

4: 96.7%

可以看到每个类别的预测精确率都在95%以上。

每个类别的召回率（将除本类别之外的其余类别看做负类别）如下：

0: 98.88%

1: 95.26%

2: 100%

3: 93.41%

4: 97.77%

可以看到召回率全部在**93%**以上。